

TAMIL NADU DEPARTMENT OF AGRICULTURE — PROTOTYPE



TN Agriculture Welfare Schemes Portal

A bilingual (Tamil + English) scheme-discovery website with an AI-powered Retrieval-Augmented-Generation (RAG) chatbot

Technical documentation describing how the application was designed and built, file by file — covering the data layer, the FastAPI + RAG backend, and the two interchangeable frontends (a plain HTML/CSS/JS site and a Streamlit app).

Generated 2 August 2026 · Repository: W1_W2_Buildathon_TNFarmersSchemeFinderApp

Contents

1. Project Overview
2. Project Structure
3. Architecture & Request Flow
4. The Data Layer — `data/schemes.json`
5. The Backend — `backend/main.py` (FastAPI)
6. The RAG Engine — `backend/rag.py`
7. Frontend #1 — `web/` (plain HTML / CSS / JS)
8. Frontend #2 — `frontend/` (Streamlit)
9. Configuration & Dependencies
10. How to Run the App
11. Design Decisions & Notes for Production

1. Project Overview

The **TN Agriculture Welfare Schemes Portal** is a bilingual (Tamil + English) web application that helps farmers in Tamil Nadu discover government agriculture welfare schemes — what benefits they offer, who is eligible, and which officer to contact. It was built as a Buildathon project and follows the structure of a real (if simplified) government citizen-services portal.

The three headline capabilities are:

- **Scheme directory** — every scheme from the Tamil Nadu Department of Agriculture is stored as structured bilingual data and rendered as browsable cards, grouped by category, with a searchable detail view.
- **RAG chatbot with memory** — a farmer can ask a free-text question ("What subsidy is there for drip irrigation?") in Tamil or English. The app retrieves the most relevant scheme(s) from a vector database and asks an LLM to answer *using only that retrieved content*, remembering the last few turns of conversation so follow-up questions ("who do I contact for it?") still work.
- **Two interchangeable frontends** — a production-style static HTML/CSS/JS site served directly by the backend, and an alternative pure-Streamlit app — both calling the exact same API, so either can be swapped in without touching the backend.

Why two frontends? The assignment allowed either a general web UI or a Streamlit app. Rather than choosing one, the project ships both, sharing one backend and one dataset, so the grader/user can run whichever is preferred.

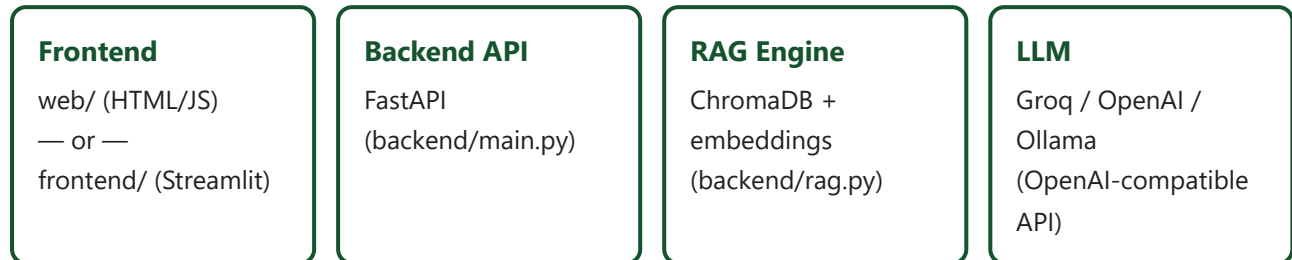
2. Project Structure

```
tn_farmers_schemes_app/
├── data/
│   └── schemes.json          # All scheme content (Tamil + English), ~22 schemes
├── backend/
│   ├── main.py              # FastAPI app – API endpoints + serves the website
│   └── rag.py               # RAG engine: embeddings + ChromaDB retrieval + LLM call
├── web/
│   ├── index.html          # Frontend #1 – plain HTML/CSS/JS (served by FastAPI)
│   ├── styles.css          # Page shell (home view, detail view, chat widget)
│   └── app.js               # Government-style visual theme
│                               # Data fetching, language toggle, chat memory
├── frontend/
│   ├── app.py              # Frontend #2 – Streamlit alternative
│   ├── translations.py     # Streamlit website (tabs, sidebar chat)
│   └── .streamlit/config.toml # All UI labels in Tamil + English
│                               # Streamlit theme colours
├── requirements.txt        # Python dependencies for backend + Streamlit
├── .env.example            # Template for LLM API keys / backend URL
├── .env                    # (git-ignored) actual local secrets
└── README.md               # Setup instructions
```

The `backend/` folder is the only part both frontends depend on — it owns the data, the search index and the LLM call, so business logic lives in exactly one place.

3. Architecture & Request Flow

The system follows a classic **client** → **API** → **RAG engine** → **LLM** pipeline. Both frontends are pure clients of the same FastAPI backend; neither talks to ChromaDB or the LLM directly.



Startup sequence

1. FastAPI starts and loads `data/schemes.json` into memory.
2. A `startup` event calls `rag.get_collection()`, which opens (or builds) a persistent ChromaDB collection on disk at `backend/chroma_store/`.
3. If the collection is empty, every scheme is flattened into one bilingual text block and embedded using a multilingual sentence-transformer model, so the first user request is fast.

Browsing flow

1. Frontend calls `GET /api/schemes` to render the home page cards / navigation list.
2. Clicking a card calls `GET /api/schemes/{id}` for the full bilingual detail (benefits, eligibility, officer, districts).

Chat (RAG) flow

1. Farmer types a question; the frontend sends it plus the conversation `history` to `POST /api/chat`.
2. `rag.answer_question()` appends the last two user turns to the question (so follow-ups like "who do I contact for it?" still retrieve the right scheme), embeds the combined text, and queries ChromaDB for the top-4 nearest scheme documents.
3. The retrieved scheme text becomes the *context* in a system + user prompt sent to the LLM, instructing it to answer only from that context, in the farmer's chosen language, and to recommend contacting the local Agricultural Officer if the answer isn't in the context.
4. The LLM's answer plus the list of source scheme ids are returned to the frontend, which shows the answer and appends "Related schemes: ..." using the titles.
5. If no LLM API key is configured, the backend still works — it simply returns the retrieved scheme text directly, so the app is testable with zero external cost.

4. The Data Layer — `data/schemes.json`

This file is the single source of truth for all scheme content and was produced by extracting and translating scheme information from the Department of Agriculture's reference material. It is a JSON array of roughly two dozen scheme objects, each with a consistent bilingual schema:

Field	Purpose
<code>id</code>	Stable slug used as the primary key everywhere (URLs, ChromaDB ids, icon lookups)
<code>category_en</code> / <code>category_ta</code>	Scheme grouping, e.g. "Increasing Crop Productivity" / பயிர் உற்பத்தியை அதிகரித்தல்
<code>title_en</code> / <code>title_ta</code>	Scheme name in both languages
<code>summary_en</code> / <code>summary_ta</code>	One-paragraph description shown on cards
<code>benefits_en[]</code> / <code>benefits_ta[]</code>	Bullet list of monetary/material benefits (subsidy rates, per-hectare amounts, etc.)
<code>eligibility_en[]</code> / <code>eligibility_ta[]</code>	Who qualifies — district restrictions, reservation quotas (SC/ST, women farmers), etc.
<code>officers_en[]</code> / <code>officers_ta[]</code>	Escalation chain of officers to contact (Village → Block → District)
<code>districts_en</code>	Free-text note on geographic applicability

Keeping both languages side-by-side in *one* record (rather than two separate files) guarantees the Tamil and English versions of a scheme can never drift out of sync, and lets the RAG engine index both languages in a single document so a question asked in either language can retrieve the same scheme.

The README explicitly flags that this Tamil text was translated for the prototype and should be verified against official department translations before any real deployment.

5. The Backend — `backend/main.py` (FastAPI)

`main.py` is a small FastAPI application that plays two roles: it is the **JSON API** consumed by both frontends, and it **serves the static website** (the `web/` folder), so a single process (`uvicorn`) is enough to run the whole product.

Startup

`data/schemes.json` is read once into a Python list (`SCHEMES`) and indexed into a dictionary by id (`SCHEMES_BY_ID`) for O(1) detail lookups. CORS middleware is enabled with a wildcard origin (`allow_origins=["*"]`) so the Streamlit frontend, which runs on a different port (8501), can call the API on port 8000 during development.

Endpoints

Method	Path	Behaviour
GET	<code>/api/health</code>	Returns <code>{"status":"ok","scheme_count":N}</code> — a quick liveness check
GET	<code>/api/schemes</code>	Returns the "short" fields of every scheme (id, category, title, summary) — enough to build cards/navigation without shipping the full bilingual payload
GET	<code>/api/schemes/{scheme_id}</code>	Returns the full scheme record; 404 if the id is unknown
POST	<code>/api/chat</code>	Body: <code>{question, lang, history}</code> . Validates the question is non-empty, converts the Pydantic history models to plain dicts, and delegates to <code>rag.answer_question()</code>

Request/response models

Pydantic models enforce the chat contract: `ChatMessage` (a single `role` + `content` turn) and `ChatRequest` (`question`, `lang` defaulting to `"ta"`, and an optional `history` list — empty for a first question). This gives free request validation and auto-generated interactive docs at `/docs`.

Serving the website

The static files in `web/` are mounted at `/static` via `StaticFiles`, and a catch-all `GET /` route returns `web/index.html` as a `FileResponse`. Because these routes are declared *after* all `/api/*` routes, API paths always take priority and the browser can call relative URLs like `/api/schemes` with no CORS or extra port needed.

6. The RAG Engine — `backend/rag.py`

This module implements the "Retrieval-Augmented Generation" pattern in four stages: index, retrieve, augment, generate.

6.1 Embeddings — understanding meaning in two languages

A multilingual sentence-transformer model, `paraphrase-multilingual-MiniLM-L12-v2`, is loaded via ChromaDB's `SentenceTransformerEmbeddingFunction`. It runs locally (no API call, no cost) and converts any text — Tamil or English — into a numeric vector, so a Tamil question can match an English-labelled scheme and vice versa.

6.2 Building the vector index

`scheme_to_text()` flattens one scheme dict into a single text block containing the title, category, summary, benefits, eligibility, districts and officers *in both languages*. `get_collection()` opens a `chromadb.PersistentClient` pointed at `backend/chroma_store/`; if the collection is empty it embeds and stores every scheme's text with its `id` as the document id, so the index survives server restarts and is only rebuilt when the store folder is deleted (e.g. after editing `schemes.json`).

6.3 Retrieval

`retrieve(question, top_k=4)` embeds the incoming query and asks ChromaDB for the four nearest scheme documents by vector similarity, returning `(id, text)` pairs.

6.4 Conversation memory

The clever part for usability: before retrieving, `answer_question()` takes the last two *user* turns from `history` and concatenates them with the new question to form the `retrieval_query`. This means a bare follow-up like "who is eligible for it?" — which on its own contains no scheme name — still retrieves the right scheme, because the previous question ("What subsidy is there for drip irrigation?") is folded into the search text.

Separately, up to `MAX_HISTORY_MESSAGES = 10` of the raw conversation turns are also passed to the LLM as prior chat messages, so the model itself can resolve pronouns like "it" or "that scheme" when composing its answer.

6.5 Generation (the LLM call)

The app talks to any **OpenAI-compatible** chat completion endpoint via the official `openai` Python SDK, configured entirely through three environment variables:

Variable	Purpose	Example
<code>LLM_BASE_URL</code>	API base URL	Groq / OpenAI / local Ollama
<code>LLM_API_KEY</code>	Auth key	Groq key, OpenAI key, or a placeholder for Ollama

LLM_MODEL	Model name	llama-3.3-70b-versatile , gpt-4o-mini , qwen2.5:0.5b
-----------	------------	--

This provider-agnostic design means swapping Groq for OpenAI or a fully offline Ollama model requires editing only `.env` — no code changes.

The system prompt constrains the model to (a) reply only in the farmer's selected language, (b) use *only* the retrieved scheme context — never invented facts, (c) use conversation history solely to resolve references, and (d) politely defer to the local Agricultural Officer when the context doesn't contain an answer. Temperature is set low (`0.2`) to keep answers factual rather than creative.

If `LLM_API_KEY` is unset, `llm_client` stays `None` and `answer_question()` gracefully falls back to returning the raw retrieved scheme text — so the whole app (search, browsing, even the chat UI) is testable with zero API keys and zero cost.

A `__main__` self-test block (`python rag.py`) demonstrates exactly this memory behaviour: it asks about drip-irrigation subsidy, then asks the follow-up "Who is eligible for it?" using the first exchange as history.

7. Frontend #1 — `web/` (Plain HTML / CSS / JS)

This is the recommended, production-style frontend: no build step, no framework — just three static files served directly by FastAPI, communicating with relative `/api/...` URLs.

7.1 `index.html` — page shell

A single HTML file holds every "view" as a `<div>`, toggled with a `hidden` CSS class rather than doing real client-side routing:

- **Top strip** — government branding + a தமிழ்/English language switch.
- **Header** — emblem, portal title, a "Home" nav button.
- **Hero section** — badge, heading, description, and a row of farmer photo thumbnails.
- `#home-view` — the scheme card grid.
- `#detail-view` — a two-column layout: a searchable, scrollable left navigation list and a right-hand content panel for the selected scheme.
- **Floating chat** (`.fab` + `.chat`) — present in the DOM on every view, so the conversation persists as the user browses.
- **Footer** — disclaimer text.

7.2 `app.js` — all behaviour

Organised into clearly numbered sections, mirroring the comments in the source file:

1. **UI text tables** (`T.ta` / `T.en`) — every static label the page needs, in both languages; scheme content itself always comes from the API, not this file.
2. **Photos** — a small table of free, openly-licensed Wikimedia Commons photo URLs, one per scheme category, used for card thumbnails and the hero background. Every `<img>` has an `onerror` handler that hides the image and falls back to a coloured gradient tile with an emoji icon, so a network hiccup never breaks the layout.
3. **State** — `lang` (persisted to `localStorage`, defaulting to Tamil), the in-memory `schemes` list, the currently open `current` scheme id, and `chatHistory` (the RAG memory array sent with every chat request).
4. **Language switching** — `setLang()` re-renders all visible text and re-fetches the detail view without a page reload.
5. **Hero photo loading** — pre-tests the hero background image before applying it, so a slow or blocked image never leaves a broken-image icon.
6. **Card rendering** — builds the home grid from the `/api/schemes` response, using each scheme's category to pick an icon / gradient / photo.
7. **Detail rendering** — the left list supports live search (filtering by title/category in either language) and highlights the active scheme; the right panel fetches `/api/schemes/{id}` and renders summary, benefits, eligibility, officers and districts as simple lists.

8. **Chat** — `sendChat()` posts the question plus the full `chatHistory` array to `/api/chat`, shows a "typing..." placeholder while waiting, then appends the bot's answer with a "Related schemes: ..." line built by mapping returned source ids back to titles. History is capped at the last 12 messages client-side to bound payload size.

7.3 `styles.css` — the visual theme

A hand-rolled government theme built around CSS custom properties: deep field green (`--field: #14532d`), harvest gold (`--gold: #d4a017`), a maroon top strip, and a warm paddy-cream page background. Notable techniques:

- A responsive card grid (`repeat(auto-fill, minmax(250px, 1fr))`) that reflows from multiple columns down to one on narrow screens.
- A sticky left navigation sidebar on the detail page (`position: sticky`) that stays visible while the right content scrolls.
- A fixed-position floating action button + chat panel, layered above all content with `z-index`, that expands via a `.open` class toggle.
- A single `@media (max-width: 760px)` breakpoint collapsing the two-column detail layout to one column, and a `prefers-reduced-motion` query that disables transitions for accessibility.

8. Frontend #2 — frontend/ (Streamlit)

An alternative, pure-Streamlit implementation of the same product, useful when the assignment specifically calls for Streamlit. It talks to the identical backend API via the `requests` library — no backend code is duplicated.

8.1 `translations.py`

A single dictionary `T = {"ta": {...}, "en": {...}}` holding every UI label (headings, button text, placeholders, error messages). Centralising labels here — rather than scattering literal strings through `app.py` — is what makes the whole UI translatable by editing one file, and is analogous to the `T` object inside `web/app.js`.

8.2 `.streamlit/config.toml`

Sets the Streamlit theme (`primaryColor = "#14532d"` to match the site's field-green brand — this is what makes the "selected scheme" navigation button appear highlighted — plus background/text colours) and `headless = true` so the server doesn't try to auto-open a browser tab.

8.3 `app.py`

Session state (`st.session_state`) tracks the current language, page (`"browse"` or `"detail"`), selected scheme id, and chat history across Streamlit's rerun-on-every-interaction model. Key design choices called out in the file's own comments:

- **Tabs instead of one long page** — a horizontal radio button styled as tabs (`st.radio(..., horizontal=True)`) switches between "Browse" and "Scheme details", keeping each screen short.
- **No hot-linked images on cards** — tiles use a CSS gradient plus an emoji icon per scheme (a hand-picked `ICON` dict keyed by scheme id, and a `TILE` dict keyed by category), so nothing can fail to load over a slow connection.
- **Chat lives in the sidebar** — the one screen region Streamlit keeps in a fixed place across reruns. A small CSS override (`flex-direction: row-reverse` on the app container) optionally moves the sidebar to the right, controlled by a single `CHAT_ON_RIGHT` flag.
- **Detail page layout** — mirrors the `web/` version: a left column of collapsible `st.expander` category groups (only the selected scheme's category auto-expands) with a search box, and a right column showing the full scheme record.
- **Chat memory** — before calling the backend, the full visible chat history (minus the canned welcome message) is converted into `{"role", "content"}` dicts and sent as `history`, exactly matching the contract expected by `POST /api/chat`.

Because both frontends send the same request shape to the same endpoints, the RAG memory behaviour described in Section 6.4 works identically regardless of which UI the farmer uses.

9. Configuration & Dependencies

9.1 `requirements.txt`

Package	Role
fastapi, uvicorn	Web framework and ASGI server for the backend
pydantic	Request/response validation for the chat API
python-dotenv	Loads <code>.env</code> into environment variables
chromadb	Local persistent vector database for retrieval
sentence-transformers	Runs the multilingual embedding model locally
openai	OpenAI-compatible client used to call Groq / OpenAI / Ollama
streamlit	Powers the alternative <code>frontend/</code> app
requests	HTTP client the Streamlit app uses to call the FastAPI backend
httpx==0.27.2	Pinned explicitly to avoid a breaking-change incompatibility with the OpenAI client

9.2 `.env.example` / `.env`

A template file documents three provider presets — **Groq** (default, fast, generous free tier), **OpenAI**, and a fully local **Ollama** model — of which only one block should be uncommented at a time. A separate `BACKEND_URL` variable tells the Streamlit frontend where the FastAPI backend lives (defaulting to `http://localhost:8000`), and `ANONYMIZED_TELEMETRY=False` disables ChromaDB's telemetry ping. The real `.env` (containing actual keys) is excluded from git via `.gitignore`.

10. How to Run the App

1. Create and activate a virtual environment, then `pip install -r requirements.txt`.
2. Copy `.env.example` to `.env` and paste an LLM API key (or leave it unset to run in fallback mode).
3. Start the one server needed: `cd backend && uvicorn main:app --reload --port 8000`. The first run downloads the embedding model and builds the ChromaDB index.
4. Open `http://localhost:8000/` — the HTML/JS site (Frontend #1) opens in Tamil by default.
5. *Optional:* with the backend still running, open a second terminal, `cd frontend && streamlit run app.py`, and open `http://localhost:8501` for the Streamlit alternative (Frontend #2).

Interactive API documentation is auto-generated by FastAPI at `http://localhost:8000/docs`, and `http://localhost:8000/api/health` gives a one-line liveness check without touching either frontend.

11. Design Decisions & Notes for Production

- **Single backend, dual frontend** — all scheme data and RAG logic live in one FastAPI service; both UIs are thin clients, so there is exactly one place to fix bugs or add schemes.
- **Bilingual-by-construction data model** — every scheme record carries Tamil and English fields side by side, so nothing can be translated for one UI and forgotten in the other.
- **Fails safe, not silent** — missing photos fall back to icon tiles, a missing LLM key falls back to raw retrieved text instead of erroring, and a lost backend connection shows a translated error message rather than a blank screen.
- **Provider-agnostic LLM layer** — swapping Groq / OpenAI / a local Ollama model is a `.env` edit, not a code change, because the app only relies on the OpenAI-compatible chat completions shape.
- **Known limitations flagged in the README, to fix before a real deployment:**
 - The Tamil translations are prototype-quality and should be verified by the department.
 - `allow_origins=["*"]` in CORS should be locked down to the real production domain.
 - Farmer/header photos are hot-linked from Wikimedia Commons; an offline deployment should download them into `web/images/` and update the URLs in `app.js`.
 - New schemes can be added by appending entries to `schemes.json` — the `backend/chroma_store/` folder must then be deleted once so the vector index rebuilds with the new content.